Datto SIRIS registration workflow

Observed on October 10, 2026

Overview and scope

This document maps the locally observable registration workflow in Datto SIRIS OS 2 5.6.3: page load, the stock network/update behavior, the optional storage pane, the required name/admin/configure/review panes, and final submission. It also records the offline-replica override that removes the network/update gate.

The browser calls JSON-RPC methods on the appliance itself. The appliance's PHP services perform the server-side work. Browser behavior is described directly from readable JavaScript and Twig templates. Server behavior is marked as inferred where the implementation is encoded.

What was examined

- **spf-datto-pkgs (10.0.3.200):** the unregistered replica. Most code reading was done there to keep load off the production appliance.
- **spf-datto (10.0.3.147):** the registered appliance. It was used only for comparisons such as file names, feature-flag state, and `upgradectl` behavior.

The two systems had the same `datto-siris-os-2` package version and the same relevant web application layout. That establishes a useful comparison point; it does not prove that every unpackaged file is byte-identical.

Workflow at a glance

Order	Pane or action	Condition	Browser-visible behavior
1	Page load	Always	Server chooses the panes and embeds session/CSRF data.
2	Network check	Stock workflow only	Retries every five seconds until connectivity succeeds.
3	Update	Stock server-selected	Starts an OS upgrade and polls progress once per second.
4	Storage	Server-selected	Inspects drives and creates the selected storage pool.
5	Name	Always	Collects the device name.
6	Administrator	Always	Collects admin credentials and rates password strength.
7	Configure	Always	Loads the default offsite location and collects configuration.
8	Review	Always	Presents the values before submission.
9	Submit	Always	Calls `send` with the completed wizard data.

```
page load
   |
   v
session + CSRF context
   |
   v
network check ---- retry every 5 s until successful
   |
   +---- optional update ---- attemptUpgrade -> getProgress loop
   |
   +---- optional storage --- checkStorage -> createStorage
   |
   v
name -> administrator -> configure -> review -> send
```

The server decides whether the update and storage panes exist. The browser builds only the panes present in the rendered page. On the air-gapped replica, setup forces the update-pane switch off and suppresses the network poll when no online-only pane remains:

```
offline page load -> name -> administrator -> configure -> review -> send
```

Where the code lives

Registration is implemented in `/usr/lib/datto/device` (part of the `datto-siris-os-2` package) and its web root, `/datto/web`. Paths below are relative to `/usr/lib/datto/device` unless they begin with `/`.

Role	File	Readable
Route `/register`	`config/routes.yaml` (`registration`)	Yes
Route `/api`	`config/routes.yaml` (`api`)	Yes
Page controller	`src/App/Controller/Web/Registration/RegisterController.php`	Encoded
Wizard API endpoint	`src/App/Controller/Api/V1/Device/Register.php`	Encoded
Registration services	`src/Service/Registration/RegistrationService.php`, `Registrant.php`	Encoded
Feature framework	`src/Feature/FeatureService.php`, `Context.php`, `DeviceRole.php`, `Features/`	Encoded
Wizard page	`templates/commonlookandfeel/Registration/Register/index.html.twig`	Yes
Wizard panes	`update`, `storage`, `name`, `admin`, `configure`, `review` pane templates	Yes
Wizard logic	`/datto/web/commonlookandfeel/js/App/Registration/RegisterWizard.js`	Yes
Wizard framework	`/datto/web/commonlookandfeel/js/Widget/Wizard3.js`	Yes
API client	`/datto/web/commonlookandfeel/UI/scripts/classes/Datto/API/Client.js`	Yes

Role	File	Readable
Shell feature library	`/datto/scripts/feature_utils`	Yes
Flag recalculation	`datto-feature-flag-update.service` and `.timer`	Yes
Cloud flag refresh	`datto-cloud-feature-refresh.service` and `.timer`	Yes
OS upgrade tool	`/usr/local/bin/upgradectl`	Root-only binary

Browser-to-device API

Wizard actions are JSON-RPC 2.0 POST requests to `/api?<method>` on the appliance. The method is also present in the JSON request body. `RegisterWizard.js` creates the API client, and the client sends `{jsonrpc: "2.0", method, params, ...}` with jQuery.

The page embeds a CSRF token and token identifier. Requests without a valid session and token are rejected with HTTP 403. The update pane uses the loss of the old session during a reboot as one signal while it waits for the appliance to return.

Method	Called when	Purpose
`testNetworkConnection`	Page load, then every five seconds	Gate network-dependent panes.
`attemptUpgrade`	Opening the update pane	Start the OS upgrade.
`getProgress`	Every second during upgrade	Read running and success state.
`checkStorage`	Storage pane	Inspect available drives.
`createStorage`	Storage pane	Create the pool on the selected drive.
`getDefaultLocation`	Configure pane	Load the default offsite datacenter.
`v1/device/security/getPasswordStrength`	Administrator pane	Rate the proposed password.
`send`	Finish	Submit the completed workflow.

The wizard also loads `/register?devicetable` with a plain AJAX GET to render the storage pane's drive table.

What happens when the page loads

1. **The server renders the page.** The encoded controller supplies switches such as `updatePaneEnabled` and `storagePaneEnabled`, plus `showDatacenters`, `storageCapacity`, session/CSRF values, and navigation URLs.
2. **The template chooses the panes.** Update and storage are added only when their server-provided switches are true. Name, administrator, configure, and review are always added.
3. **The browser builds the wizard.** `RegisterWizard` inherits from `Wizard3` and attaches validation and load/unload hooks to each pane.

4. **The stock network check starts immediately.** A successful response hides the warning and sets `networkConnected = true`. Other results show the warning and schedule a retry five seconds later. The stock update pane cannot advance while the flag is false.

The rules that select optional panes are server-side and encoded. Their exact conditions should not be inferred solely from variable names.

Update behavior

Opening the update pane calls `attemptUpgrade`. While an upgrade is active, the browser calls `getProgress` once per second and reads `isUpgradeRunning` and `wasUpgradeSuccessful`. A reboot invalidates the original web session, so transient HTTP 403 responses are expected while the appliance restarts.

The browser-side polling behavior is confirmed. Package selection, rollback behavior, and the internal actions performed by `/usr/local/bin/upgradectl` remain server-side or binary implementation details and require separate observation.

Offline replica override

The air-gapped setup modifies both registration templates so `updatePaneEnabled = false` before the pane list and browser context are built. It also changes `RegisterWizard` to call `checkNetworkConnectivity()` only when an authorization or update pane is present. With both online-only panes absent, the wizard sets `networkConnected = true` locally and begins at **Name the Device**.

This override fixes an inconsistency in the stock failure path: `setUpgradeError()` visually enables **Next**, but the update pane's validator still rejects the click while `networkConnected === false`. Production Twig caches are invalidated during setup so the offline template takes effect immediately.

Storage and final configuration

The storage pane calls `checkStorage` to populate the device table and `createStorage` for the selected drive. The later panes collect the device name, administrator settings, offsite location, and remaining configuration. The review pane presents the completed values before `send` submits them.

The precise server-side persistence order is not visible in the encoded PHP. A failed or interrupted run should therefore be evaluated through logs and before/after state, not inferred from the browser sequence alone.

Feature flags

The installation contains roughly 154 feature classes, plus local recalculation and cloud-refresh timers. The readable shell helper exposes flag values to scripts, while the PHP service determines flags for the web application.

This document records only directly observed flag state. A useful follow-up is a registered-versus-unregistered comparison containing the flag name, both values, source, refresh mechanism, and observed effect.

Evidence boundaries and open questions

- Browser request order, pane construction, polling intervals, and client-side gates are directly observable.
- PHP controller and service internals are encoded; their effects require logs, network traces, or before/after state to confirm.
- The exact destination tested by `testNetworkConnection` was not established from the browser code alone.
- The conditions controlling the update and storage panes need controlled-state testing.
- Upgrade rollback and interrupted-storage behavior still need explicit failure tests.